



Integrating Vertical and Horizontal Partitioning into Automated Physical Database Design

Sanjay Agrawal

Microsoft Research

sagrawal@microsoft.com

Vivek Narasayya

Microsoft Research

viveknar@microsoft.com

Beverly Yang*

Stanford University

byang@stanford.edu

Submitted by:

Yashasvi Ishwarbhai Jadav (202301069)

Anushka Prajapati (202301224)

B.Tech ICT, DA-IICT

Submitted to:

P. M. Jat

Big Data Processing

Dhirubhai Ambani University (DAU)

Date: November 5, 2025

Contents

1	Abstract	2
2	Motivation and Objectives	2
2.1	a. Problem that the Paper Addresses	2
2.2	b. Why the Problem is Important	2
3	Related Background	3
4	Summarize the Proposed Solution	4
4.1	Architecture of the Solution	4
4.1.1	Column-Group Restriction (Novel Pre-Processing Step)	4
4.1.2	Candidate Selection	5
4.1.3	Merging	5
4.1.4	Enumeration (Final Design Selection)	5
4.2	Key Innovations and Contributions	6
5	Evaluation of the Proposed Solution	6
5.1	Experimental Setup: Dataset and Workload	6
5.2	Results and Discussion	7
5.2.1	Superiority of the Integrated Approach	7
5.2.2	Effectiveness of Column-Group Pruning	7
5.2.3	Criticality of Co-Location in Merging	8
5.3	Benchmarking and Alignment of the Results	9
6	Final Conclusions and Takeaways from the study	9
7	Any Future directions they suggest and you think!	10
7.1	Future directions suggested by the authors	10
7.2	Future directions (my own reflections)	10
8	References	10

1 Abstract

This report analyzes the 2004 SIGMOD paper "Integrating Vertical and Horizontal Partitioning into Automated Physical Database Design" by Agrawal, Narasayya, and Yang. The study introduces a unified and scalable framework that integrates the selection of indexes, materialized views, vertical partitioning (VP), and horizontal partitioning (HP) into a single, automated physical database design process. Unlike prior work that treated these components separately, this approach jointly optimizes for query performance and manageability, specifically addressing the alignment constraint between tables and indexes. Using cost-based pruning and a Lazy Enumeration technique, the proposed system efficiently searches large design spaces while maintaining high-quality results. This report summarizes the paper's motivation, its proposed solution and architecture, and its experimental evaluation, concluding with key takeaways and potential future research directions.

2 Motivation and Objectives

2.1 a. Problem that the Paper Addresses

The paper tackles the challenge of integrating horizontal and vertical partitioning into the process of automated physical database design. Traditionally, automated tuning tools for relational databases have focused on indexes and materialized views, but have not included partitioning decisions within the same optimization framework.

However, horizontal and vertical partitioning critically affect performance and manageability:

- **Horizontal partitioning** divides tables and indexes into disjoint row subsets (using range or hash functions), influencing how efficiently data can be accessed and maintained.
- **Vertical partitioning** divides tables into disjoint column subsets, reducing the amount of data scanned during queries.

The main problem is that partitioning choices interact strongly with other physical design elements (like indexes and materialized views). Optimizing them independently often leads to inferior performance because:

- Separately chosen indexes and partitions may duplicate effort or conflict in their effects on queries.
- The design space grows combinatorially when both vertical and horizontal partitioning are considered.
- Manageability constraints, such as alignment requirements (keeping indexes and tables partitioned identically), further complicate the search for an optimal configuration.

Thus, the paper addresses how to automatically select and optimize a combination of partitioning, indexing, and materialized view strategies in a single, integrated framework that balances both performance and manageability.

2.2 b. Why the Problem is Important

This problem is important because physical design choices directly determine database performance, scalability, and maintainability.

Performance Impact:

- As shown in the paper's TPC-H query example, treating partitioning separately from indexing can lead to 30% slower query execution.
- Integrated design enables the optimizer to exploit interdependencies between indexes and partitions, reducing sorting and data-access costs.

Manageability and Administrative Efficiency:

- DBAs often need aligned partitions for indexes and tables to simplify backup, restore, and maintenance operations.

- Incorporating alignment ensures that physical design remains operationally practical, not just performant.

Scalability Challenge:

- Modern enterprise workloads involve huge numbers of queries and tables, making manual design infeasible.
- The combinatorial explosion of design alternatives (due to multiple partitioning types and column arrangements) demands automated, cost-based pruning and search techniques.

Research Gap:

- Previous automated design systems (e.g., AutoAdmin) optimized indexes and views but ignored partitioning and manageability constraints.
- This paper is among the first to propose a scalable, integrated physical design system that handles both.

In summary, the study is motivated by the need to bridge the gap between high-performance design and real-world database manageability, offering an integrated and scalable approach to physical design automation.

3 Related Background

The paper builds on several key physical database design concepts:

- **Vertical Partitioning (VP):** This technique splits a table into multiple "sub-tables," each containing a disjoint set of *columns*.

	A	B	C
1	A1	B1	C1
2	A2	B2	C2
3	A3	B3	C3

	A	B
1	A1	B1
2	A2	B2
3	A3	B3

	C
1	C1
2	C2
3	C3

The key columns of the original table are typically replicated in each sub-table to allow for the reconstruction of the original rows. This improves performance for queries that only access a small subset of a table's columns.

- **Horizontal Partitioning (HP):** This technique splits an access method (like a table, index, or materialized view) into disjoint sets of *rows*. These disjoint sets are stored and accessed separately.

id	A	B	C
1	A1	B1	C1
2	A2	B2	C2
3	A3	B3	C3
4	A4	B4	C4

id	A	B	C
1	A1	B1	C1
2	A2	B2	C2

id	A	B	C
3	A3	B3	C3
4	A4	B4	C4

The two common types are:

- **Range Partitioning:** Rows are partitioned based on a range of values in a specific column (e.g., partitioning sales data by date).

- **Hash Partitioning:** A hash function is applied to one or more columns to determine which partition a row belongs to.
- **Alignment:** This is a key manageability requirement. It mandates that indexes on a table must be horizontally partitioned in the *exact same way* as the table itself.
- **The Physical Design Problem:** The goal is to find a **configuration** (a valid set of physical design structures) that minimizes the total optimizer-estimated cost for a given workload W , subject to a total storage budget S . This problem, and its sub-problems (like index selection and partitioning), are NP-hard.

4 Summarize the Proposed Solution

The paper presents a cost-based, integrated architecture for automating physical database design, incorporating horizontal partitioning, vertical partitioning, indexes, and materialized views in a unified framework. Unlike staged approaches that optimize each component separately, the proposed system jointly selects and configures all design elements to achieve optimal performance while ensuring manageability through alignment (i.e., ensuring indexes and tables share the same partitioning scheme).

The goal is to find a physical design configuration that minimizes query execution cost for a given workload under storage and alignment constraints, using a scalable, optimizer-driven search.

4.1 Architecture of the Solution

The architecture extends Microsoft’s AutoAdmin framework[1] and is built around four sequential stages (as shown in Figure 1). Each stage contributes to reducing complexity, enhancing scalability, and improving overall solution quality.

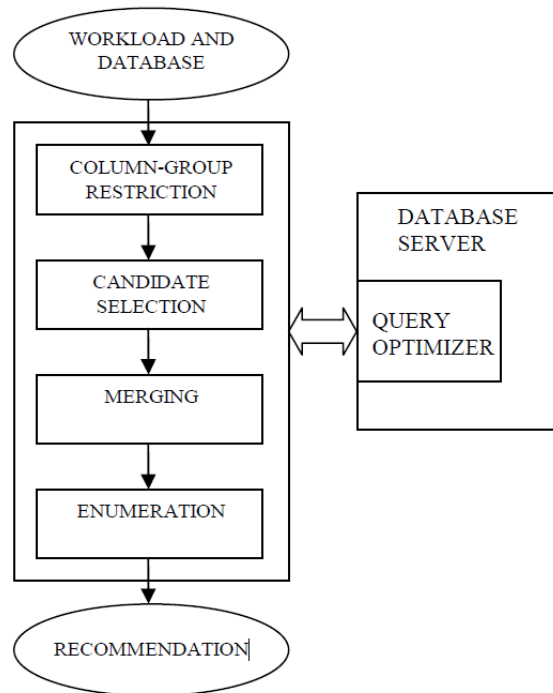


Figure 1: Architecture of the proposed integrated physical design system.

4.1.1 Column-Group Restriction (Novel Pre-Processing Step)

Goal: Control the combinatorial explosion of potential design structures.

Method:

- Identifies only “interesting” column groups that significantly influence workload performance.
- Introduces a metric **CG-Cost(g)** — the fraction of total workload cost referencing column-group g — and retains only those groups above a threshold f .
- Exploits the monotonicity of CG-Cost and uses frequent-itemset generation algorithms to prune unimportant column combinations efficiently.
- Adds a new metric **VPC (Vertical Partitioning Confidence)** to measure how strongly columns co-occur across queries. High-VPC column groups are prioritized for vertical partitioning since splitting them would incur costly joins.

This step significantly reduces the search space while maintaining near-optimal design quality.

4.1.2 Candidate Selection

Goal: Identify high-quality physical design structures for each individual query.

Method:

- Uses the “interesting” column groups (ranked by VPC) to generate per-query design candidates — indexes, vertical and horizontal partitions, and materialized views.
- Employs a cost-based evaluation using the database’s query optimizer (“what-if” analysis) to estimate performance impact without physically creating structures.
- Applies the **Greedy (m, k)** algorithm to select an optimal subset of candidates for each query while accounting for intra-query interactions and co-location (e.g., joins on the same partitioned columns).

This ensures that the best per-query configurations are efficiently identified for later consolidation.

4.1.3 Merging

Goal: Combine and generalize query-specific candidates to produce shared structures that benefit the overall workload.

Challenges: Integration of partitioning adds complexity since merged structures must preserve performance and co-location properties.

Methods:

- **Vertical Partition Merging:** Uses unions (to reduce join cost) and intersections (to minimize redundant scans) of parent sub-tables, selecting the lowest-cost configuration via a join-and-scan cost model.
- **Horizontal Partition Merging:** Simultaneously merges indexes and their partitioning methods, exploiting interchangeability between indexing and partitioning columns.
- **Range Partitioning:** Introduces a greedy algorithm, *MergeRanges*, which merges adjacent intervals to minimize total scan and partition-management cost.
- **Co-location Preservation:** Ensures merged structures retain compatible partitioning with other tables used in co-located joins; alternative merged versions are generated when necessary.

The Merging step yields a compact, workload-optimized set of shared design candidates.

4.1.4 Enumeration (Final Design Selection)

Goal: From all original and merged candidates, produce the final workload-optimized configuration that satisfies storage and alignment constraints.

Method:

- Employs the **Greedy (m, k)** search strategy for scalability.
- To enforce alignment efficiently, the paper proposes **Lazy Enumeration**, which introduces aligned versions of structures only when required — that is, when an unaligned structure is selected by the search.

- This replaces the computationally expensive **Eager Enumeration** (which pre-generates all possible aligned structures, leading to quadratic growth in complexity).

Results: Lazy Enumeration achieves up to 90% faster execution with negligible (<1%) quality loss, providing a practical balance between scalability and design accuracy.

4.2 Key Innovations and Contributions

- **Integrated Optimization:** Jointly considers partitioning, indexing, and materialized views to avoid poor staged designs.
- **Workload-Driven Pruning:** Efficiently reduces the design space using CG-Cost and VPC metrics without sacrificing accuracy.
- **Advanced Merging Algorithms:** First to handle merging of both vertically and horizontally partitioned structures while maintaining co-location.
- **Efficient Alignment Handling:** Introduces Lazy Enumeration to meet manageability constraints without exponential overhead.
- **Scalable Cost-Based Framework:** Relies on query-optimizer “what-if” interfaces instead of actual implementation, ensuring scalability for enterprise workloads.

5 Evaluation of the Proposed Solution

The efficacy of the proposed integrated physical design methodology was validated through a comprehensive experimental evaluation. The authors implemented the techniques as a prototype on a commercial database server. The evaluation was structured to quantify:

1. The performance benefits of an integrated approach versus staged (VP→IND→HP) or index-only (IND-ONLY) approaches.
2. The effectiveness of the “Column-Group Restriction” heuristic in pruning the search space.
3. The performance impact of co-location considerations during the Merging step.
4. The scalability and quality trade-offs of the “Lazy Enumeration” algorithm for handling alignment constraints.

5.1 Experimental Setup: Dataset and Workload

The experimental testbed utilized the following components:

- **Database:** The TPC-H 1GB (TPCH1G) database was the primary dataset for most experiments. An additional 1.2 GB APB database with complex queries was used for alignment-specific tests.
- **Hardware:** All experiments were conducted on a workstation equipped with a 1GHz x86 processor and 256 MB of RAM.
- **Workloads:** A diverse set of workloads was employed to analyze various facets of the system:
 - **TPCH22:** The 22 queries from the standard TPC-H benchmark.
 - **CS-WKLD:** A 100-query workload of randomly generated Select-Project-Join (SPJ) queries over the TPCH1G database.
 - **WKLD-COL-n:** A synthetic workload suite comprising 25 queries, where n (20, 40, 60, 80) denotes the percentage of queries containing join conditions. This suite was specifically designed to test co-location benefits.
 - **APB:** A 200-query workload of complex decision support queries.

5.2 Results and Discussion

The empirical results provide strong evidence for the authors’ central hypotheses.

5.2.1 Superiority of the Integrated Approach

This experiment evaluated the solution quality (based on optimizer-estimated cost) of the integrated **TOGETHER** approach against the **IND-ONLY** and staged **VP→IND→HP** methodologies, varying the available storage bound.

As illustrated in Figure 2, the non-integrated approaches exhibited significant quality degradation.

- The **IND-ONLY** approach performed poorly, especially in low-storage scenarios (up to 40% quality reduction at 1300 MB). This is attributed to its inability to leverage partitioning, which offers substantial performance gains for minimal storage cost.
- The staged **VP→IND→HP** approach was consistently inferior across all storage bounds. This result validates the paper’s core motivation: a non-integrated approach, by making “greedy” physical design choices in isolated stages, precludes superior solutions that an integrated method can discover. For example, a vertical partition chosen in the first step may render the optimal set of indexes or horizontal partitions unusable.

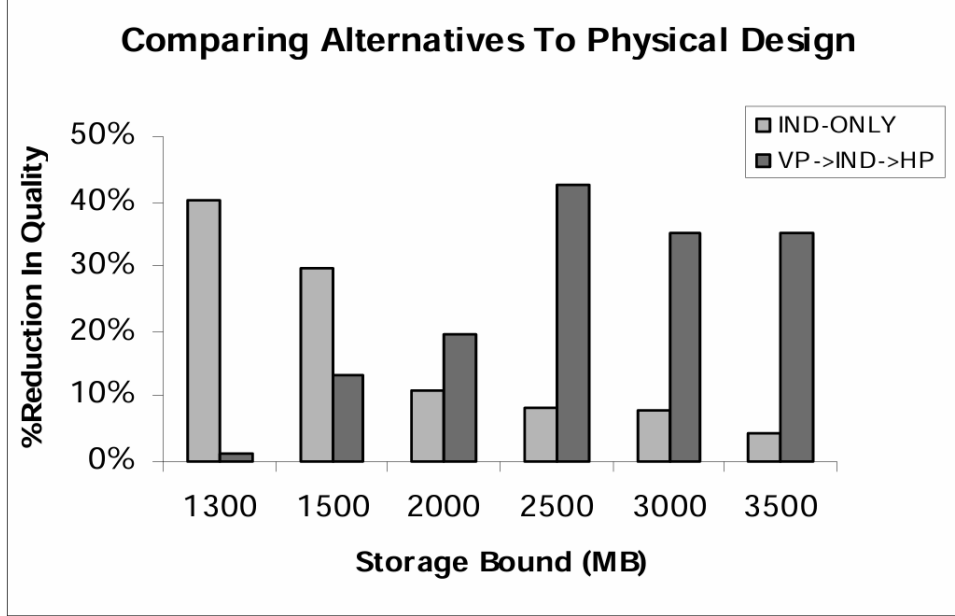


Figure 2: Comparing Alternatives To Physical Design

5.2.2 Effectiveness of Column-Group Pruning

This set of experiments measured the trade-off between solution quality and tool runtime when applying the “Column-Group Restriction” pruning heuristic, controlled by the threshold f .

- **Quality (Figure 3):** The results demonstrate that a small threshold ($f = 0.02$) induces negligible quality degradation (only 2% for TPCH22) compared to no pruning ($f = 0.0$). Quality degrades more rapidly for $f > 0.02$, particularly for the CS-WKLD, which has less locality.
- **Running Time (Figure 4):** Conversely, the impact on performance is substantial. A threshold of $f = 0.02$ achieved a runtime reduction of nearly 20% for TPCH22.

Taken together, these results confirm that the heuristic is highly effective. A threshold of $f \approx 0.02$ represents an optimal trade-off, significantly improving the tool’s scalability with minimal impact on the final solution’s quality.

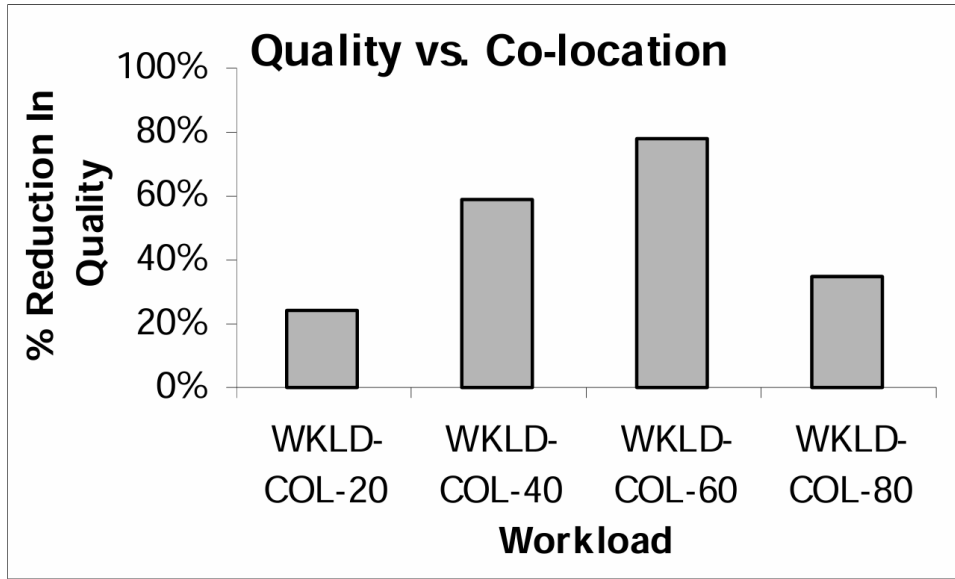


Figure 3: Quality vs. Column-Group Threshold

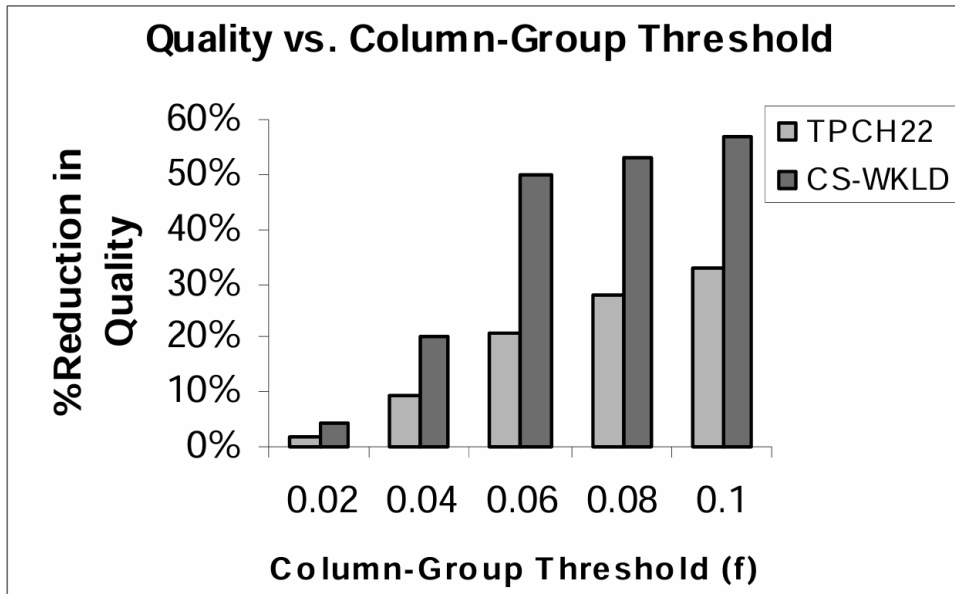


Figure 4: Running Time vs. Column-Group Threshold

5.2.3 Criticality of Co-Location in Merging

This experiment was designed to isolate the importance of co-location (aligning partition keys for joins) during the Merging step. The full algorithm was compared against a variant (**NOCOL**) that omits co-location considerations.

Figure 5 demonstrates that ignoring co-location has severe performance consequences.

- As the percentage of join queries in the workload increased (from WKLD-COL-20 to WKLD-COL-60), the quality of the NOCOL solution degraded substantially, reaching a peak quality reduction of nearly 80%.
- This confirms that co-location is a critical factor for join-heavy workloads, enabling high-performance partition-wise joins. An algorithm that fails to account for this during the Merging phase will produce a demonstrably inferior physical design.

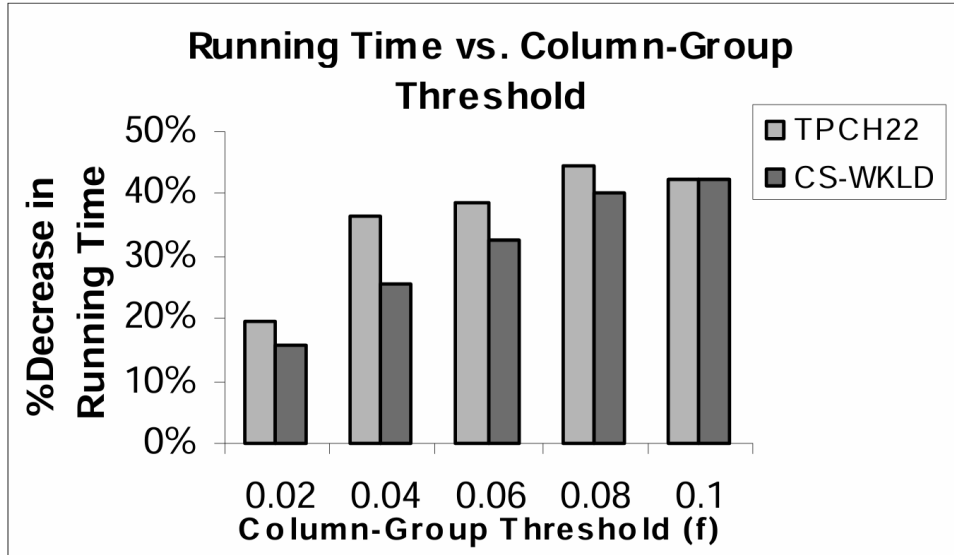


Figure 5: Quality vs. Co-location

5.3 Benchmarking and Alignment of the Results

A final experiment evaluated the authors' novel **Lazy Enumeration** algorithm, which is designed to efficiently enforce the manageability constraint of index alignment. This was benchmarked against a naive **Eager Enumeration** strategy, which pre-computes the cross-product of all indexes and partitioning methods.

The results, summarized in Table 2 (from the paper), highlight a clear win for the proposed technique:

- **Scalability:** Lazy Enumeration was significantly more scalable, reducing runtime by **90%** for the TPCH22 workload and 50% for the complex APB workload. This is because it avoids the combinatorial explosion of the eager approach by only generating aligned candidates on demand as required by the search.
- **Quality:** This dramatic performance gain was achieved with almost no compromise in quality. The solution quality was only **1% worse** for TPCH22 and **0% worse** for APB compared to the exhaustive eager method.

This benchmark confirms that Lazy Enumeration is a highly effective and scalable strategy for integrating manageability constraints into the physical design problem.

6 Final Conclusions and Takeaways from the study

This paper provides a comprehensive and scalable solution to the integrated physical database design problem.

Key Takeaways:

- **Integration is Non-Negotiable:** Physical design choices (partitioning, indexes, views) are deeply interconnected. A staged approach that considers them in isolation is guaranteed to produce suboptimal results.
- **The Search Space is Manageable:** The "combinatorial explosion" from adding partitioning can be effectively managed through intelligent, workload-based pruning techniques like the Column-Group Restriction.
- **Manageability Can Be Automated:** The "Lazy Enumeration" algorithm provides a practical and highly scalable way to incorporate complex manageability constraints (like alignment) directly into the search process without sacrificing performance.
- **Details Matter:** Seemingly small details, like preserving co-location during the merging step, can have an outsized impact on the quality of the final design for join-heavy workloads.

7 Any Future directions they suggest and you think!

7.1 Future directions suggested by the authors

The authors' work focuses exclusively on partitioning in a **single-node environment** (e.g., on a single large server). They explicitly suggest that future work should investigate how their techniques can be adapted to a **multi-node partitioning** environment (i.e., a shared-nothing cluster). This would require extending the model to consider not only performance but also **availability** requirements, which multi-node partitioning can provide.

7.2 Future directions (my own reflections)

- **Dynamic and Adaptive Design:** The paper assumes a static, representative workload is given beforehand. A significant future direction would be to adapt these design principles for dynamic environments where the workload patterns drift over time. This would require an online system that can monitor the workload and recommend or even apply partitioning changes automatically.
- **Cloud-Native Cost Models:** The entire solution relies on the query optimizer's "what-if" cost estimates. In modern cloud-native databases (like Snowflake, BigQuery, or Redshift Spectrum), storage and compute are decoupled. The cost models are entirely different, often based on bytes scanned and compute-seconds used rather than I/O. This paper's techniques would need to be fundamentally revisited to work with these new cost models and architectures.
- **Machine Learning for Pruning:** The `CG_Cost` and `VP-CONFIDENCE` metrics are effective, cost-based heuristics. However, a machine learning model could potentially be trained on workload traces and query plans to learn more complex interactions. Such a model might be able to more accurately predict the "benefit" of a partitioning strategy, leading to even more aggressive and intelligent pruning of the search space.

8 References

- [1] Agrawal, S., Narasayya, V., & Yang, B. (2004). Integrating Vertical and Horizontal Partitioning into Automated Physical Database Design. *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data (SIGMOD '04)*, 359–370.